

Theory of Computation

Presented By

Dr Saravanan S ME,Ph.D

School of Computing Science and Engineering

VIT Bhopal University, Bhopal

Theory of Automata

- Theory of automata is a theoretical branch of computer science and mathematical. It is the study of abstract machines and the computation problems that can be solved using these machines. The abstract machine is called the automata. The main motivation behind developing the automata theory was to develop methods to describe and analyse the dynamic behaviour of discrete systems.

- The **State** is represented by **circles**, and the **Transitions** is represented by **arrows**.

Symbols:

- Symbols are an entity or individual objects, which can be any letter, alphabet or any picture.

Example:

1, a, b, #

Alphabets:

- Alphabets are a finite set of symbols. It is denoted by Σ .
- Examples:
- $\Sigma = \{a, b\}$
- $\Sigma = \{A, B, C, D\}$
- $\Sigma = \{0, 1, 2\}$
- $\Sigma = \{0, 1, \dots, 5\}$
- $\Sigma = \{\#, \beta, \Delta\}$

String:

- It is a finite collection of symbols from the alphabet. The string is denoted by w .

Example 1:

- If $\Sigma = \{a, b\}$, various string that can be generated from Σ are $\{ab, aa, aaa, bb, bbb, ba, aba, \dots\}$.
- A string with zero occurrences of symbols is known as an empty string. It is represented by ϵ .
- The number of symbols in a string w is called the length of a string. It is denoted by $|w|$.
- Example 2:

$$w = 010$$

$$\text{Number of Sting } |w| = 3$$

Language:

- A language is a collection of appropriate string. A language which is formed over Σ can be **Finite** or **Infinite**.

- Example: 1

$L1 = \{\text{Set of string of length 2}\} = \{aa, bb, ba, bb\}$

$L2 = \{\text{Set of all strings starts with 'a'}\} = \{a, aa, aaa, abb, abbb, ababb\}$

Finite Automata

- Finite automata are used to recognize patterns.
- It takes the string of symbol as input and changes its state accordingly. When the desired symbol is found, then the transition occurs.
- At the time of transition, the automata can either move to the next state or stay in the same state.
- Finite automata have two states, **Accept state** or **Reject state**. When the input string is processed successfully, and the automata reached its final state, then it will accept.

Formal Definition of FA

- A finite automaton is a collection of 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where:
- Q : finite set of states
- Σ : finite set of the input symbol
- q_0 : initial state
- F : **final** state
- δ : Transition function

Finite Automata Model

- Finite automata can be represented by input tape and finite control.
- **Input tape:** It is a linear tape having some number of cells. Each input symbol is placed in each cell.
- **Finite control:** The finite control decides the next state on receiving particular input from input tape. The tape reader reads the cells one by one from left to right, and at a time only one input symbol is read.

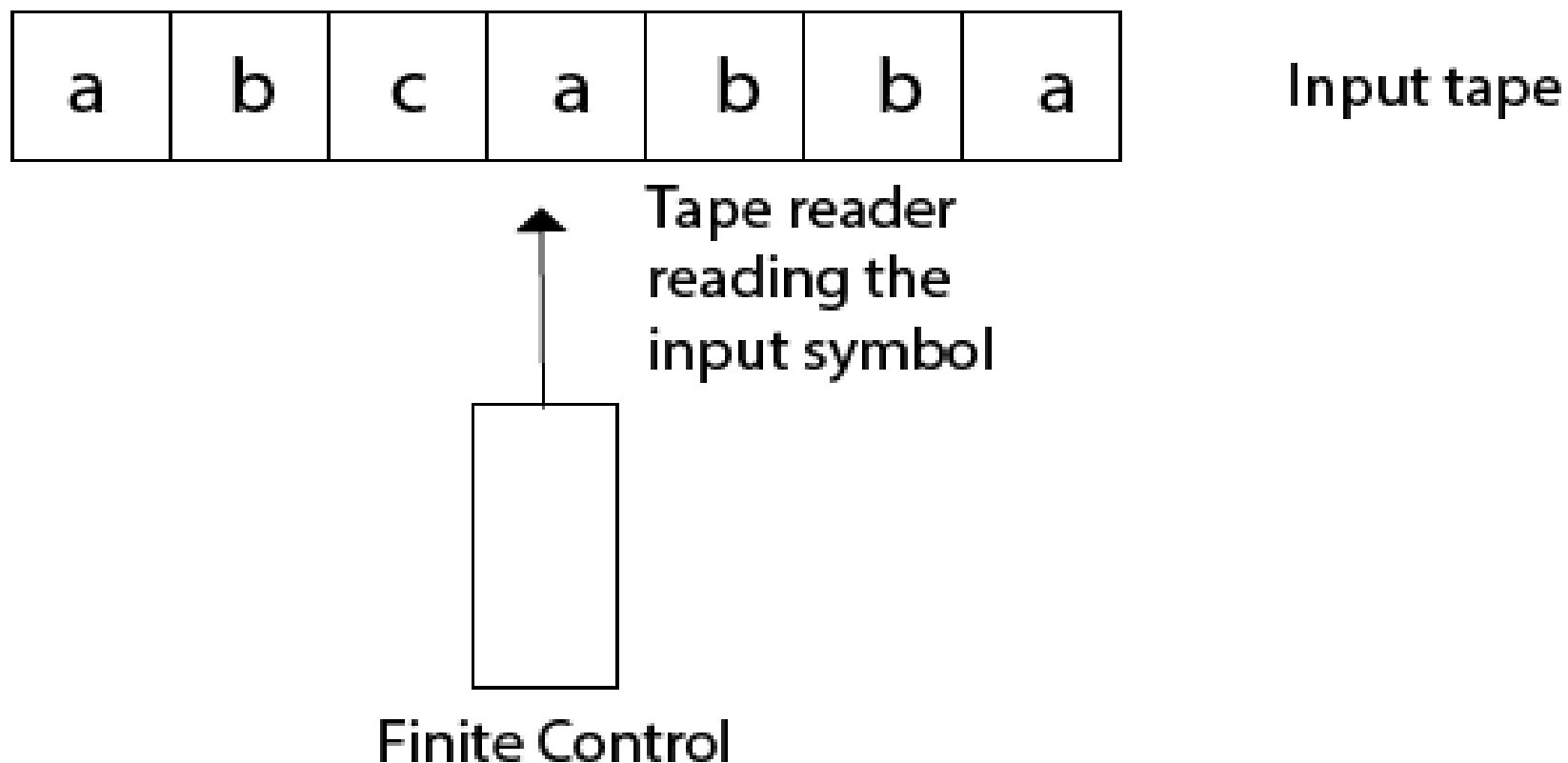


Fig :- Finite automata model

Transition Diagram

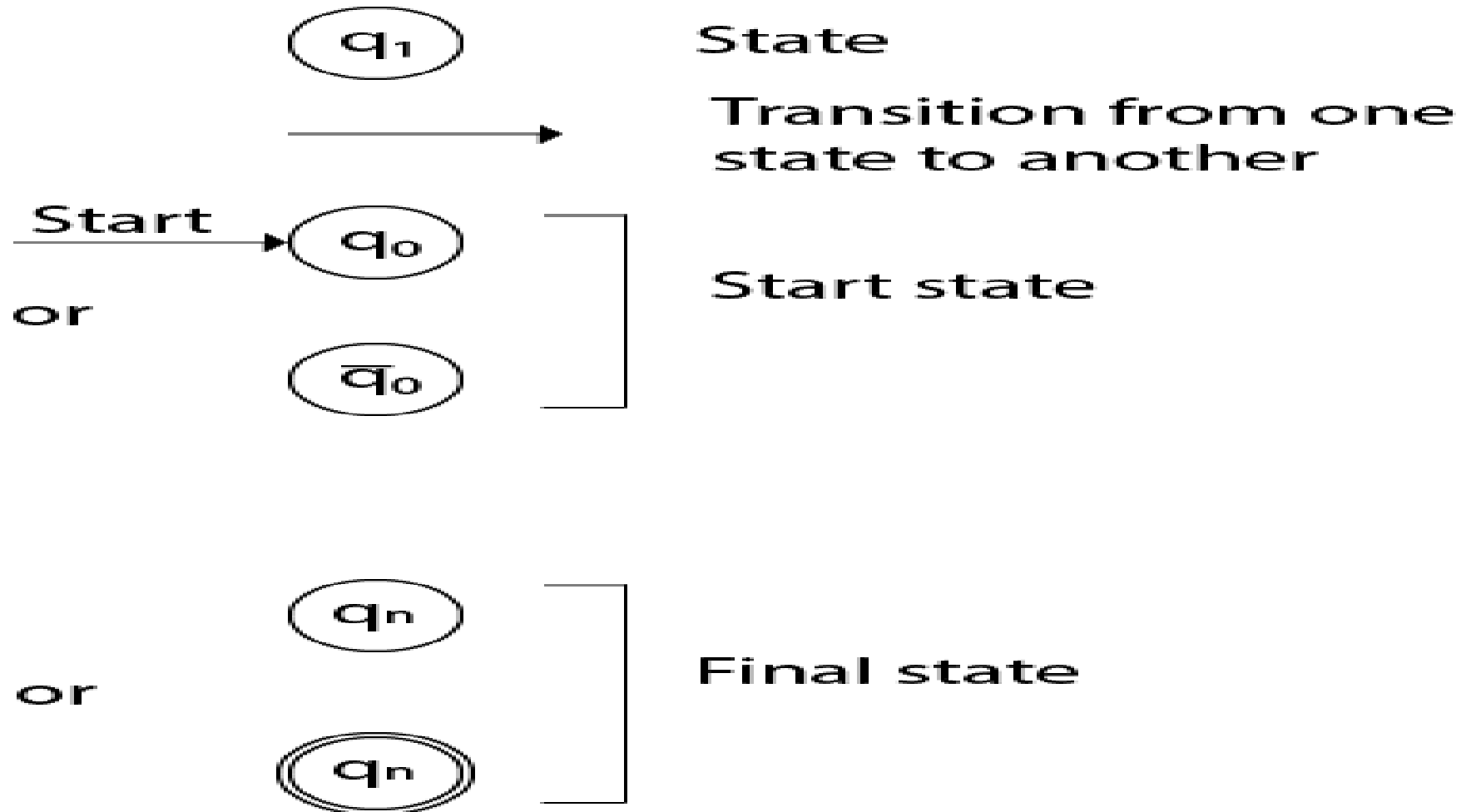
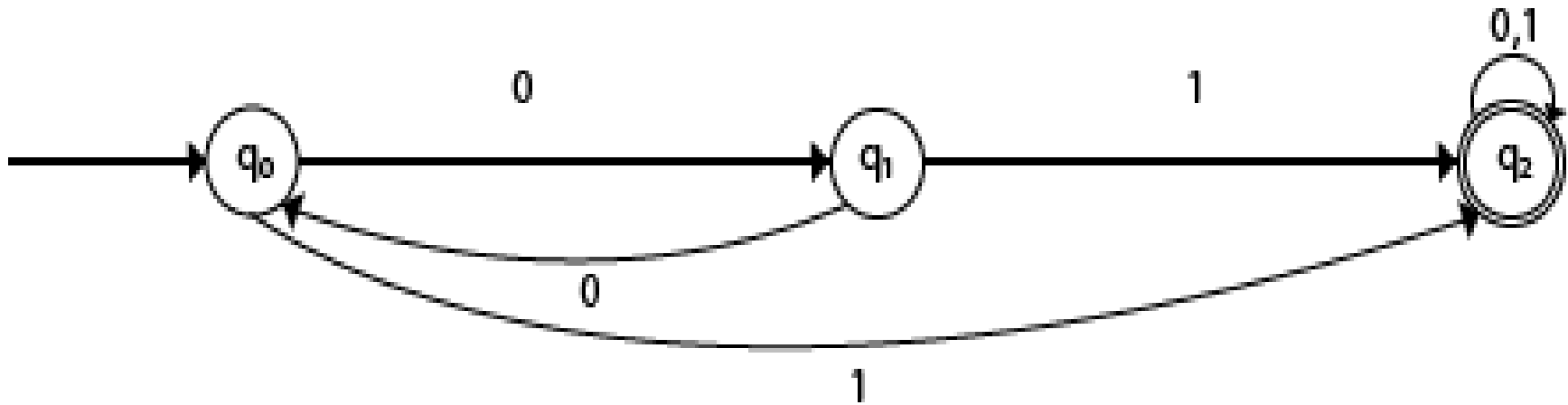


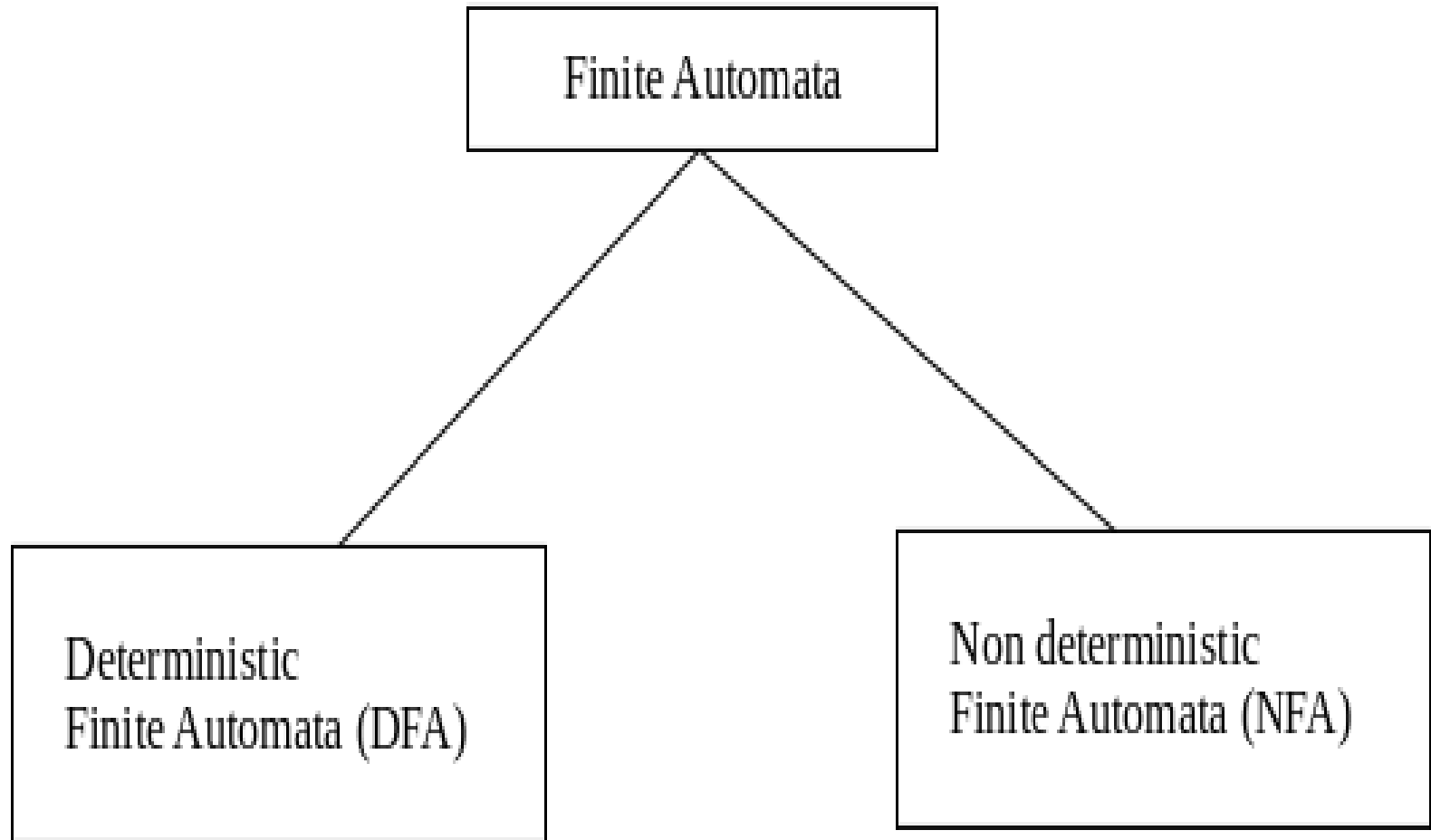
Fig:- Notations

Transition Table



Present State	Next state for Input 0	Next State of Input 1
→q0	q1	q2
q1	q0	q2
*q2	q2	q2

Types of Automata



Deterministic Finite Automata(DFA)

- DFA refers to deterministic finite automata. Deterministic refers to the uniqueness of the computation. The finite automata are called deterministic finite automata if the machine is read an input string one symbol at a time.
- In DFA, there is only one path for specific input from the current state to the next state.
- DFA does not accept the null move, i.e., the DFA cannot change state without any input character.
- DFA can contain multiple final states. It is used in Lexical Analysis in Compiler

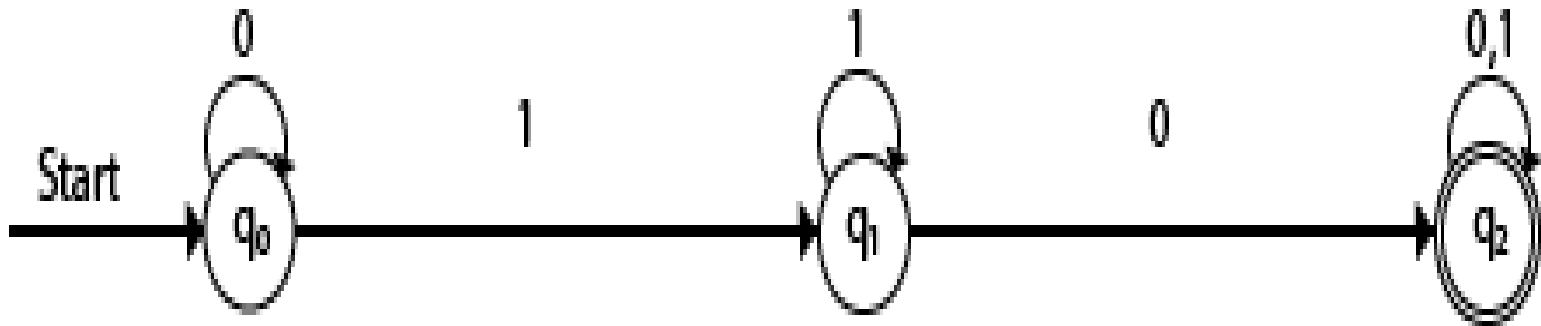
Formal Definition of DFA

A DFA is a collection of 5-tuples same as we described in the definition of FA

- Q : finite set of states
- Σ : finite set of the input symbol
- q_0 : initial state
- F : **final** state
- δ : Transition function $Q \times \Sigma \rightarrow Q$

Example 1

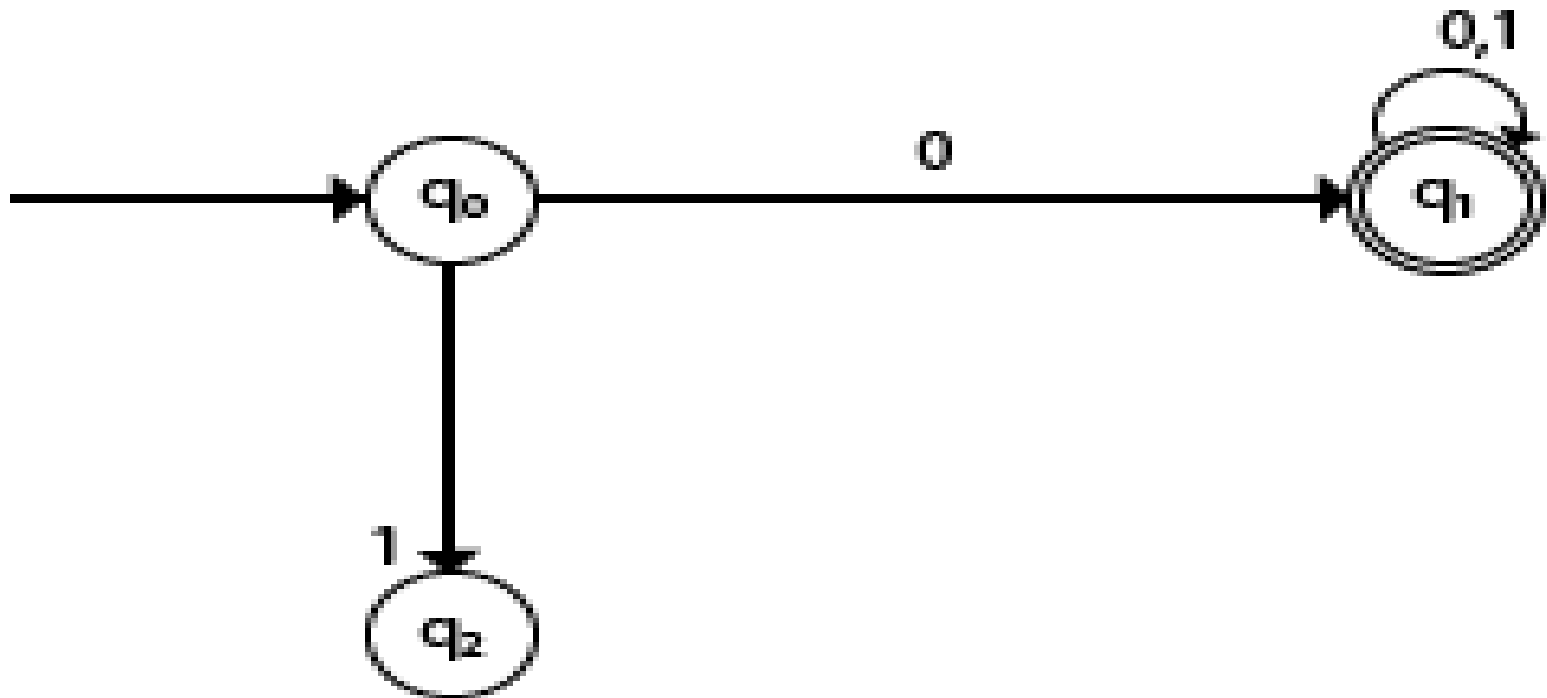
- $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{0, 1\}$,
 $\delta, q_0 = \{q_0\}, F = \{q_2\} = ?$



State	0	1
→q0	q0	q1
q1	q2	q1
*q2	q2	q2

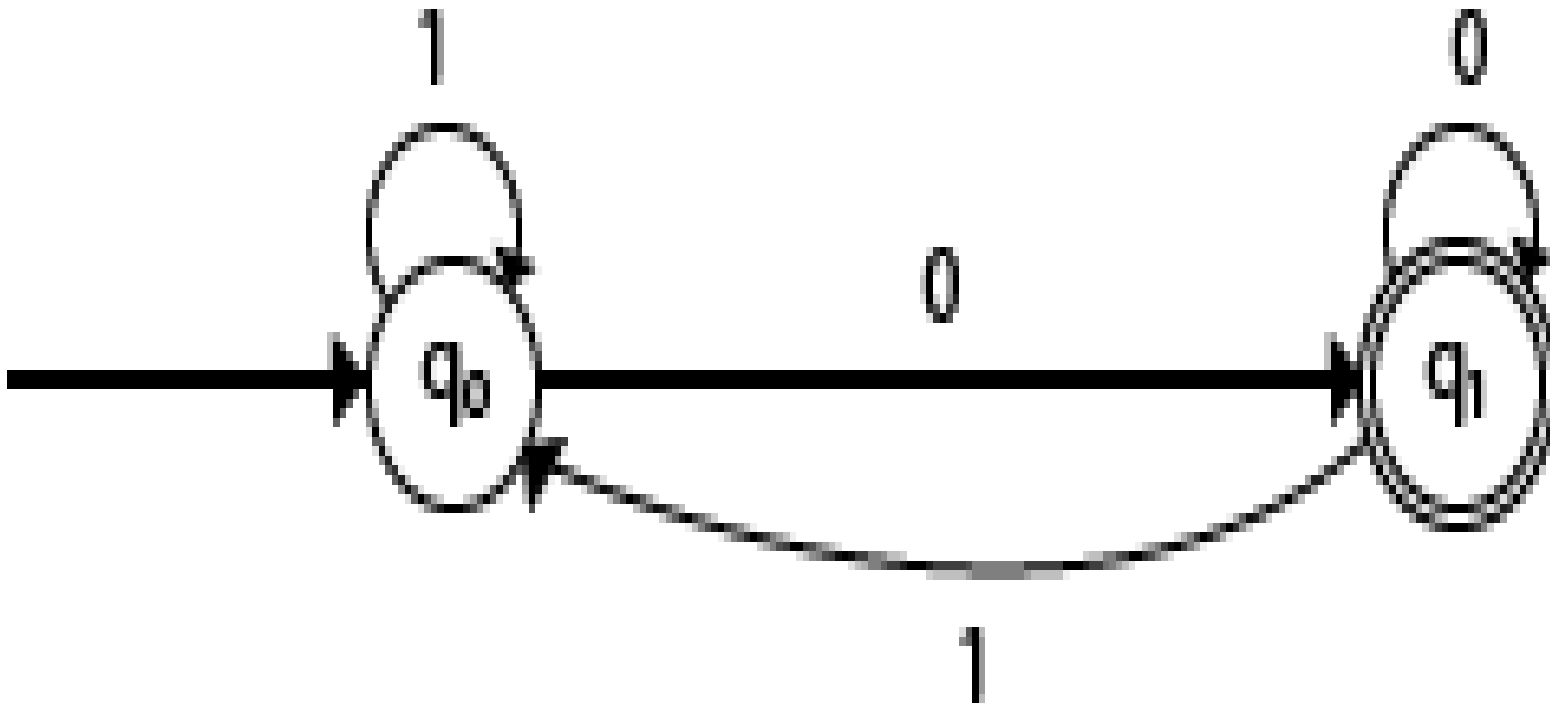
Example 2

DFA with $\Sigma = \{0, 1\}$ accepts all starting with 0.



Example 3

DFA with $\Sigma = \{0, 1\}$ accepts all ending with 0.



Non-Deterministic Finite Automata(NFA or NDFA)

- NFA stands for non-deterministic finite automata. It is easy to construct an NFA than DFA for a given regular language.
- The finite automata are called NFA when there exist many paths for specific input from the current state to the next state.
- Every NFA is not DFA, but each NFA can be translated into DFA.
- NFA is defined in the same way as DFA but with the following two exceptions, it contains multiple next states, and it contains ϵ transition.

Example

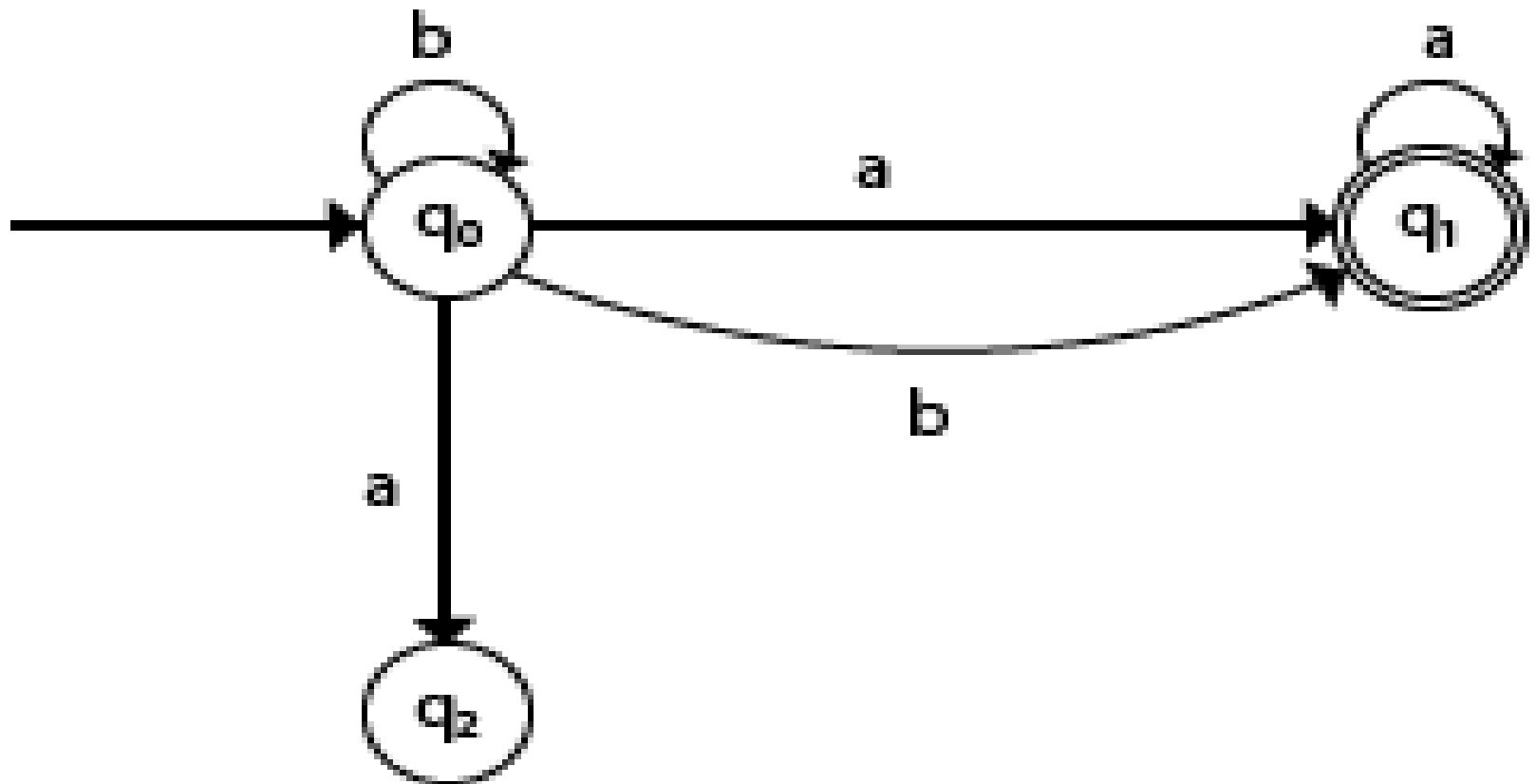


Fig:- NDFA

Formal definition of NFA

A DFA is a collection of 5-tuples same as we described in the definition of FA

- Q : finite set of states
- Σ : finite set of the input symbol
- q_0 : initial state
- F : **final** state
- δ : Transition function $\delta: Q \times \Sigma \rightarrow 2^Q$

Example

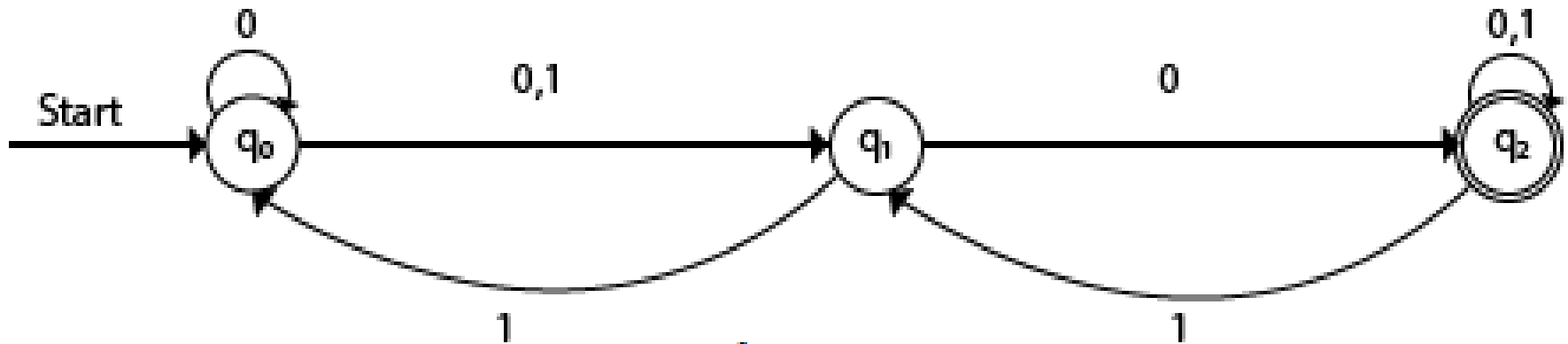


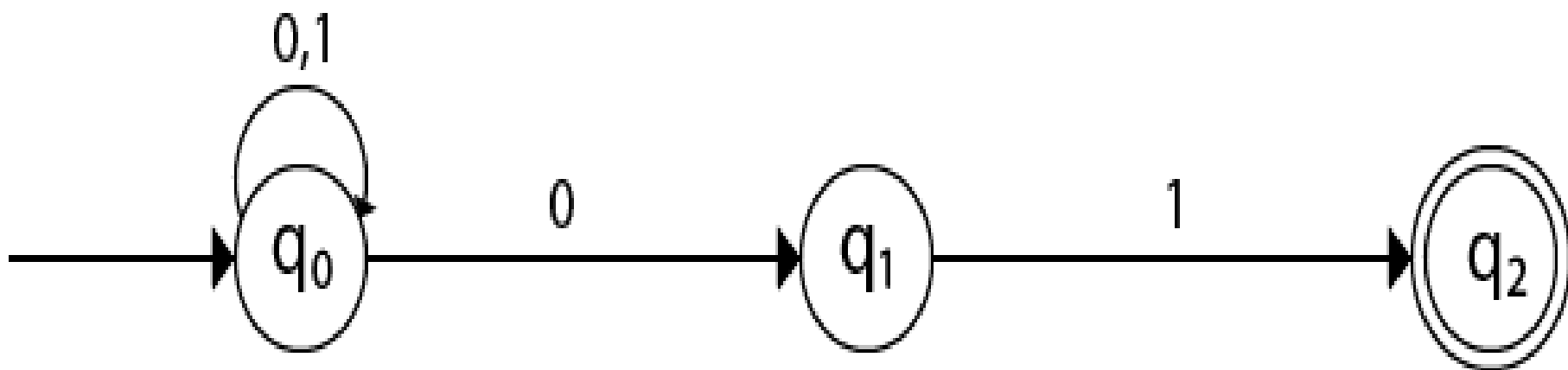
Fig: NFA

Present State	Next state for Input 0	Next State of Input 1
→q0	q0, q1	q1
q1	q2	q0
*q2	q2	q1, q2

Example 2

Present State	0	1
$\rightarrow q_0$	q_0, q_1	q_0, q_2
q_1	q_3	ϵ
q_2	q_2, q_3	q_3
$*q_3$	q_3	q_3

Example 3



Problem 1

- Let $M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, q_0)$ and the transition functions are

$$\delta(q_0, 0) = q_2 \qquad \delta(q_2, 0) = q_0$$

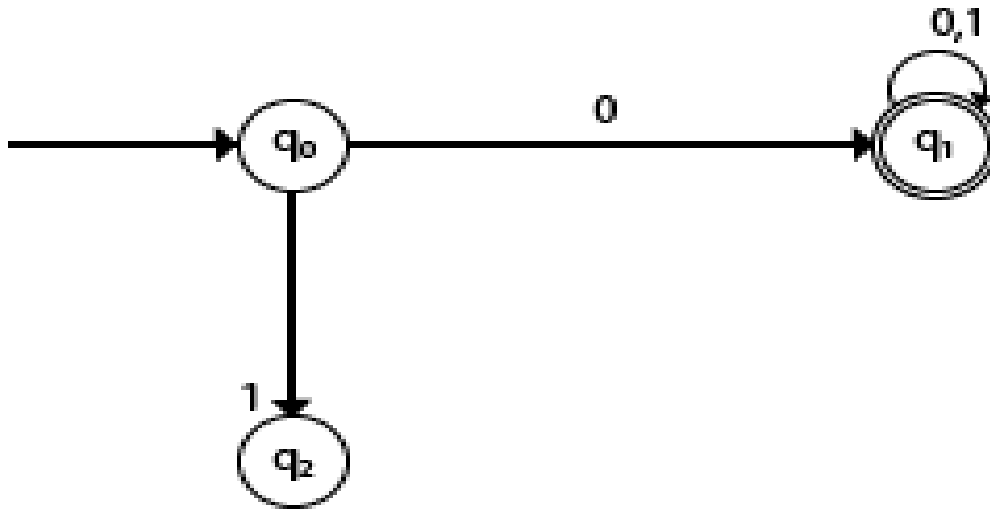
$$\delta(q_0, 1) = q_1 \qquad \delta(q_2, 1) = q_3$$

$$\delta(q_1, 0) = q_3 \qquad \delta(q_3, 0) = q_1$$

$$\delta(q_1, 1) = q_0 \qquad \delta(q_3, 1) = q_2$$

- a) Represent M by its State table
- b) Represents M by its State diagram
- c) Which of the following strings are accepted by
 - 1) 0101 2) 110101 3) 01001

Problem 2



- String are{ 001, 1001,01101}

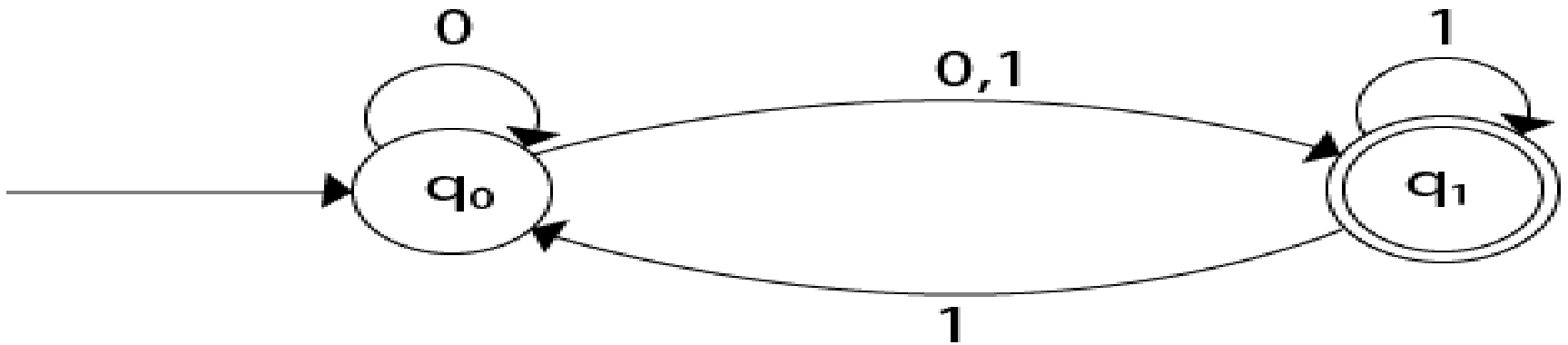
Conversion from NFA to DFA

Let, $M = (Q, \Sigma, \delta, q_0, F)$ is an NFA which accepts the language $L(M)$. There should be equivalent DFA denoted by $M' = (Q', \Sigma', q_0', \delta', F')$ such that $L(M) = L(M')$.

Steps for converting NFA to DFA

- **Step 1:** Initially $Q' = \phi$
- **Step 2:** Add q_0 of NFA to Q' . Then find the transitions from this start state.
- **Step 3:** In Q' , find the possible set of states for each input symbol. If this set of states is not in Q' , then add it to Q' .
- **Step 4:** In DFA, the final state will be all the states which contain F (final states of NFA)

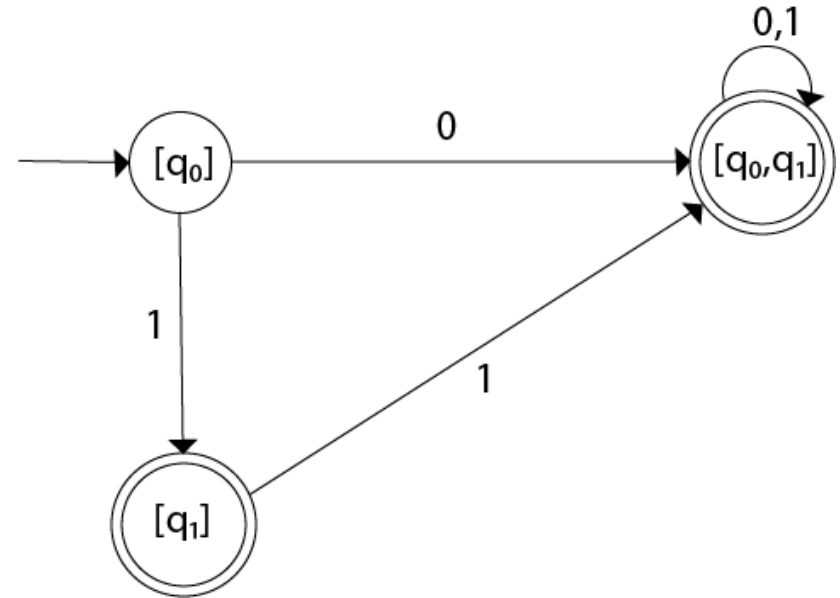
Example 1: Convert the given NFA to DFA.



State	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_1\}$
$*q_1$	ϕ	$\{q_0, q_1\}$

Solution

State	0	1
$\rightarrow[q_0]$	$[q_0, q_1]$	$[q_1]$
$*[q_1]$	ϕ	$[q_0, q_1]$
$*[q_0, q_1]$	$[q_0, q_1]$	$[q_0, q_1]$



Conversion from NFA with ϵ to DFA

Steps for converting NFA with ϵ to DFA:

Step 1: We will take the ϵ -closure for the starting state of NFA as a starting state of DFA.

Step 2: Find the states for each input symbol that can be traversed from the present. That means the union of transition value and their closures for each state of NFA present in the current state of DFA.

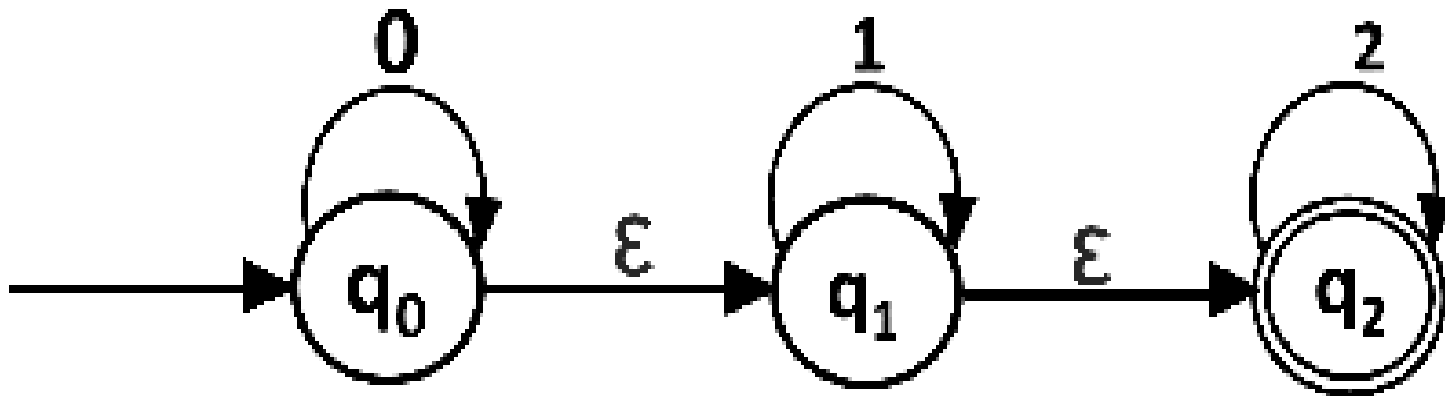
Step 3: If we found a new state, take it as current state and repeat step 2.

Step 4: Repeat Step 2 and Step 3 until there is no new state present in the transition table of DFA.

Step 5: Mark the states of DFA as a final state which contains the final state of NFA.

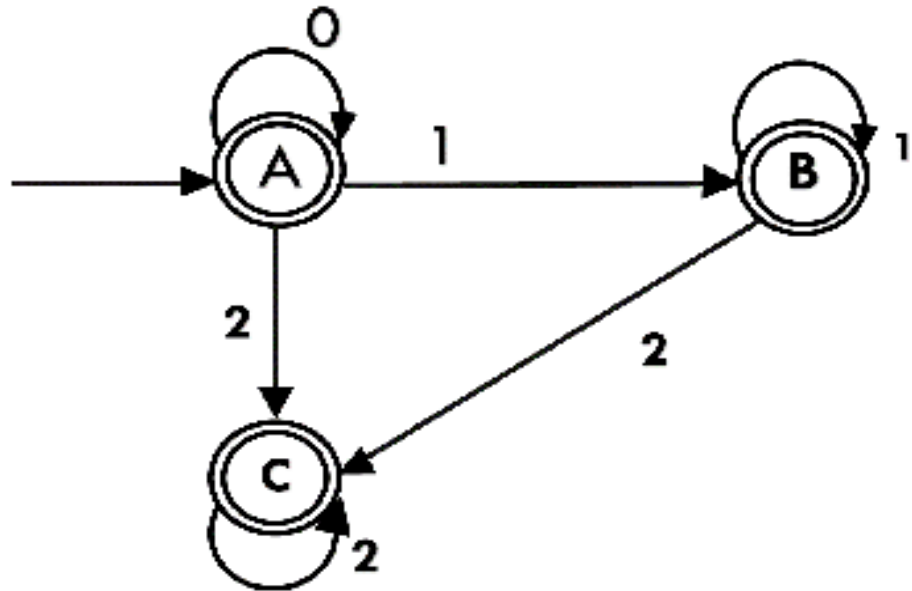
Example

Convert the given NFA into its equivalent DFA



Solution

- $\varepsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$
- $\varepsilon\text{-closure}(q_1) = \{q_1, q_2\}$
- $\varepsilon\text{-closure}(q_2) = \{q_2\}$
- $\delta'(A, 0) = A$
- $\delta'(A, 1) = B$
- $\delta'(A, 2) = C$
- $\delta'(B, 0) = \varphi$
- $\delta'(B, 1) = B$
- $\delta'(B, 2) = C$



Minimization of DFA

- **Step 1:** Remove all the states that are unreachable from the initial state via any set of the transition of DFA.
- **Step 2:** Draw the transition table for all pair of states.
- **Step 3:** Now split the transition table into two tables T1 and T2. T1 contains all final states, and T2 contains non-final states.
- **Step 4:** Find similar rows from T1 such that:

Pumping Lemma (For Regular Languages)

>> Pumping Lemma is used to prove that a Language is NOT REGULAR

>> It cannot be used to prove that a Language is Regular

If A is a Regular Language, then A has a Pumping Length ' P ' such that any string ' S ' where $|S| \geq P$ may be divided into 3 parts $S = x y z$ such that the following conditions must be true:

- (1) $x y^i z \in A$ for every $i \geq 0$
- (2) $|y| > 0$
- (3) $|xy| \leq P$

To prove that a language is not Regular using PUMPING LEMMA, follow the below steps:

(We prove using Contradiction)

- > Assume that A is Regular
- > It has to have a Pumping Length (say P)
- > All strings longer than P can be pumped $|S| \geq P$
- > Now find a string 'S' in A such that $|S| \geq P$
- > Divide S into x y z
- > Show that $x y^i z \notin A$ for some i
- > Then consider all ways that S can be divided into x y z
- > Show that none of these can satisfy all the 3 pumping conditions at the same time
- > S cannot be Pumped == CONTRADICTION

Pumping Lemma (For Regular Languages) - EXAMPLE (Part-1)

Using Pumping Lemma prove that the language $A = \{a^n b^n \mid n \geq 0\}$ is Not Regular

Proof:

Assume that A is Regular

Pumping length = p

$$S = a^p b^p \Rightarrow S = a a a a a a b b b b b b$$

$\begin{array}{c} \text{---} \\ | \quad | \quad | \\ x \quad y \quad z \end{array}$

$p=7$

Case 1: The γ is in the 'a' part

aaaaaa a bbbbbb
x y z

Case 2: The γ is in the 'b' part

aaaaaa bbbbb b
x y z

Case 3: The γ is in the 'a' and 'b' part

aaaaaa aa bbbbbb
x y z

$$xy^iz \Rightarrow xy^2z \quad \times$$

aa aaaaaaaa bbbbbb
11 $\neq$ 7

$$xy^iz \Rightarrow xy^2z \quad \times$$

aaaaaa bb bbbb bbbb b
7 $\neq$ 11

$$xy^iz \Rightarrow xy^2z \quad \times$$

aaaaa aabbaabb bbbb

$a^n b^n$

$$|xy| \leq p \quad p=7$$



Pumping Lemma (For Regular Languages) EXAMPLE (Part-2)

Using Pumping Lemma prove that the language $A = \{yy \mid y \in \{0,1\}^*\}$ is Not Regular

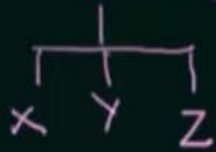
Proof:

0101

Assume that A is Regular

then it must have a pumping length = P

$$S = 0^P 1 0^P 1$$



$$P = 7$$



$$xy^iz \Rightarrow xy^2z$$

000000000000100000001

$\notin A$

$$|y| > 0$$

$$|xy| \leq P = 7$$

DECISION PROPERTIES OF REGULAR LANGUAGES:

1. EMPTINESS PROBLEM
2. FINITENESS PROBLEM
3. EQUIVALENCE PROBLEM

PROBLEM : SOLVABLE OR UNSOLVABLE

SOLVABLE = 1. DECIDABLE

2. UNDECIDABLE

DECISION PROPERTIES OF REGULAR LANGUAGES:

1. EMPTINESS PROBLEM

- DECIDABLE PROBLEM.
- To check whether a given FA accepts empty language or non-empty language.

ALGORITHM:

- Eliminate the inaccessible states from the FA
- If the resultant FA consist atleast 1 Final State.

2. FINITENESS PROBLEM

It states that the language accepted by a given FA is **finite or not**.

Algorithm:

I

- Same as emptiness problem
- Eliminate states from which final states are not reachable.
- In the resultant FA if there exist any loop or cycle, then automata accepts infinite language, otherwise finite.

EQUIVALENCE OF TWO FINITE AUTOMATA

↳ Condition is

① Two finite automata over Σ are equivalent if they accept the same set of strings over Σ .

② When two finite automata are not equivalent, there is some string w over Σ satisfying the following:



4. MEMBERSHIP PROBLEM

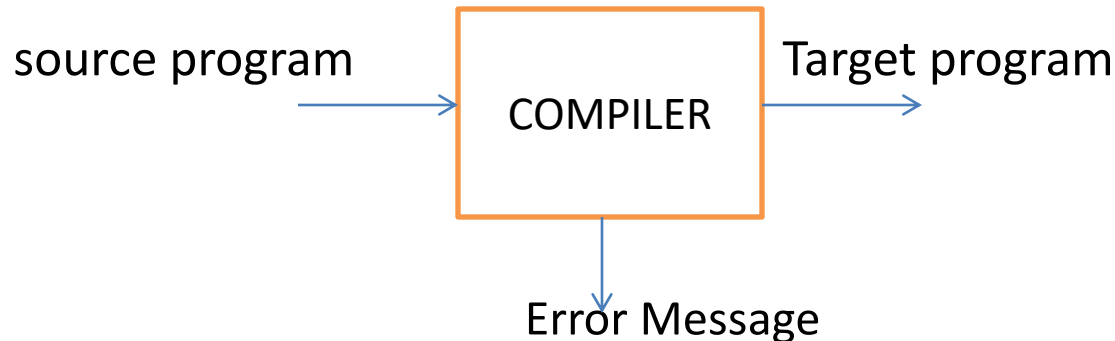
To check that a particular string is accepted or not by a given FA:

- It is decidable, because by reading the string if it reaches the final state, then it is a member.
- Otherwise, if the string halts in a non- final state then it is said to be not a member of the FA.

COMPILER

DEFINITION

- ✓ Compiler is a program that reads a program written in one language – source program – and translate it into an equivalent program in another language – target language
- ✓ Compiler reports to its user the presence of error in source program



Types of Compiler

There are 4 types

- Single – Pass
- Multi- Pass
- Load and Go
- Debugging

Analysis – Synthesis Model

There are 2 parts to compilation

1. Analysis Model

- breaks up the source program into constituent pieces and creates an intermediate representation of the source program
- Tree → Syntax tree

Analysis – Synthesis model

2.Synthesis Model

Constructs the desired target program from the intermediate representation

Software Tools :

- ✓ Structural Editors
- ✓ Pretty Printers
- ✓ Static Checkers
- ✓ Interpreters
- ✓ Text formatters
- ✓ Silicon compilers
- ✓ Query interpreters

Context of compilers

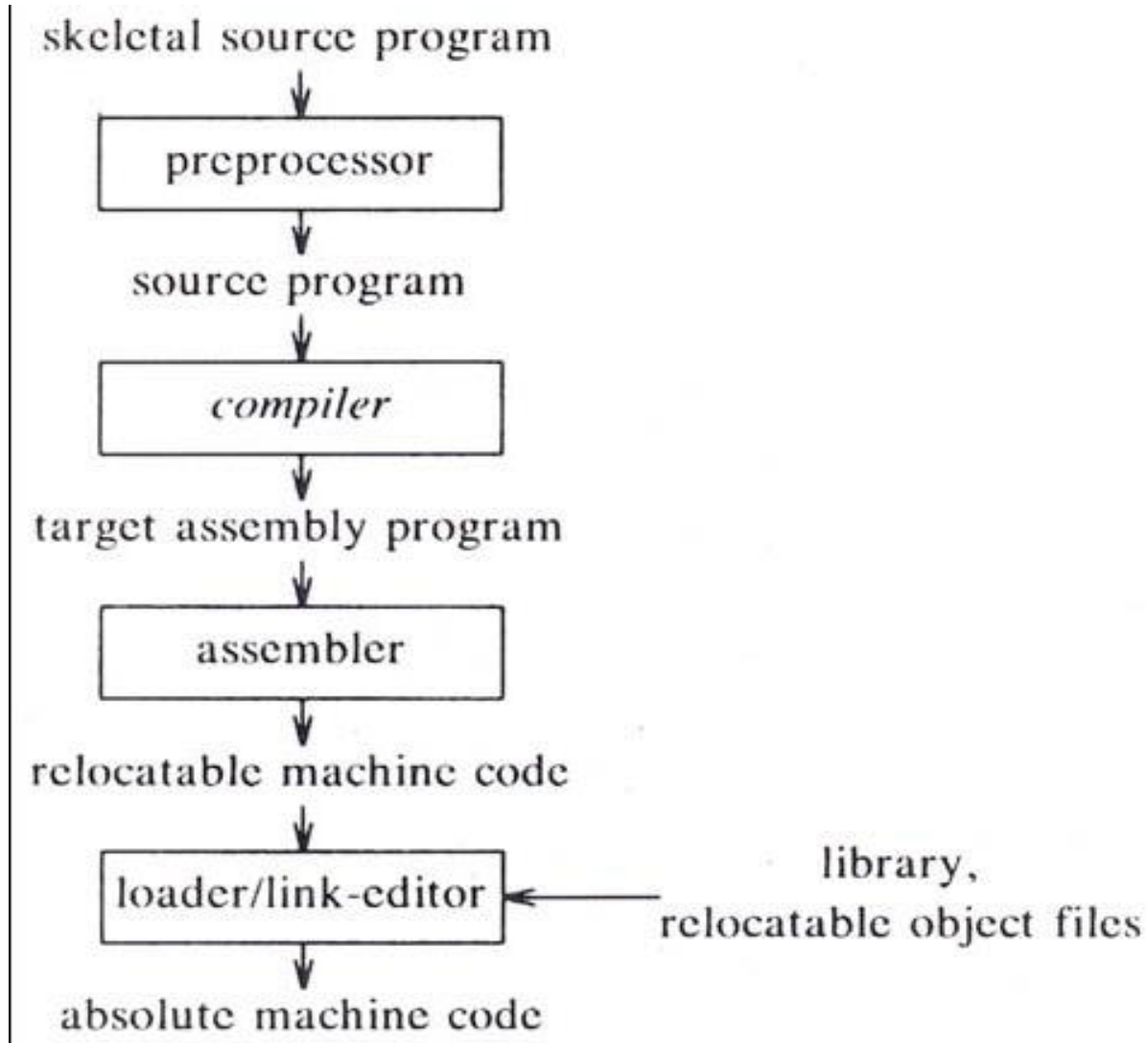


Figure 1. A Language-Processing System

Analysis of the source program

Analysis consists of 3 phases

1.Linear analysis

- source program is read from left-to-right
- Grouped into tokens → sequences of characters having a collective meaning.

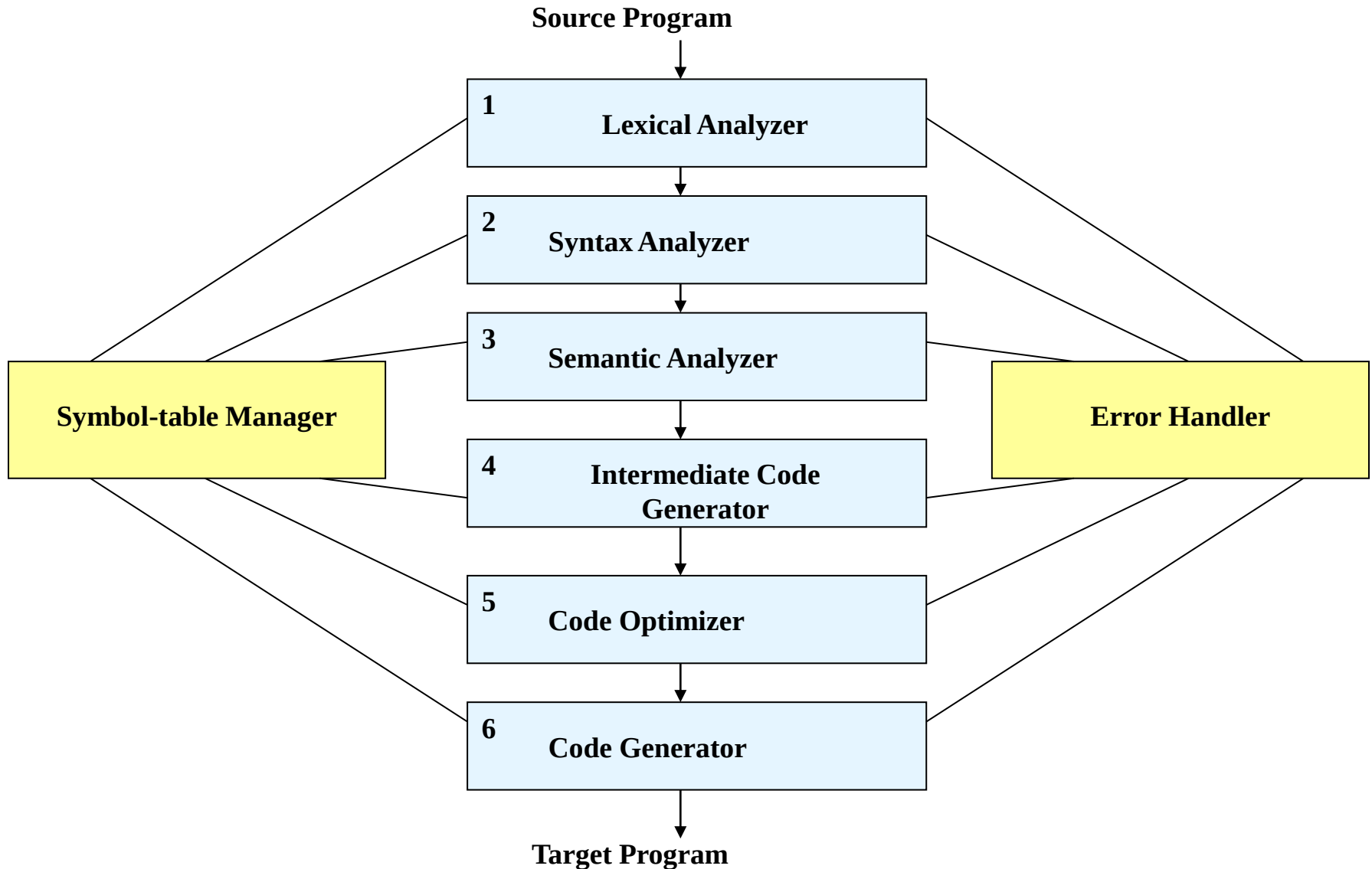
2.Hierarchical analysis

- tokens are grouped hierarchically into nested collections with collective meaning.

3.semantic analysis

- to ensure that the components of a program fit together meaningfully.

Phases of a Compiler

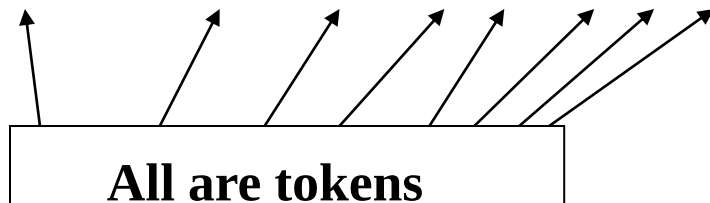


Lexical Analysis

- Also called scanning
 - Scans Input & Identify tokens
 - Removes White spaces and comments

An Example

Position := initial + rate * 60 ;



Token

- **Three types of tokens:**
- Terminal Symbols (TRM)-
- Keywords and Operators, Literals (LIT),
- Identifiers (IDN).

Example

- `int a = 10;`

Tokens

`int` (keyword),
`a` (identifier),
`=` (operator),
`10` (constant)
`;` (punctuation-semicolon)

Total token = 5

Example 2

```
int main()  
{  
    // printf() sends the string inside quotation to  
    // the standard output (the display)  
    printf("Welcome to GeeksforGeeks!"); return 0;  
}
```

- **Tokens** 'int', 'main', '(', ')', '{', 'printf', '(', ' ' 'Welcome to GeeksforGeeks!' ')', ';', 'return', '0', ';', '}'

Total number of tokens = 14

Lexeme

- It is a sequence of characters in the source code that are matched by given predefined language rules for every lexeme to be specified as a valid token. **int, goto, main**

Example

main is lexeme of type identifier(token)

(,),{,} are lexemes of type punctuation(token)

Pattern

- It specifies a set of rules that a scanner follows to create a token.
- It must start with the alphabet, followed by the alphabet or a digit.
- The sequence of characters that make the keyword.

Example

(,), {, }

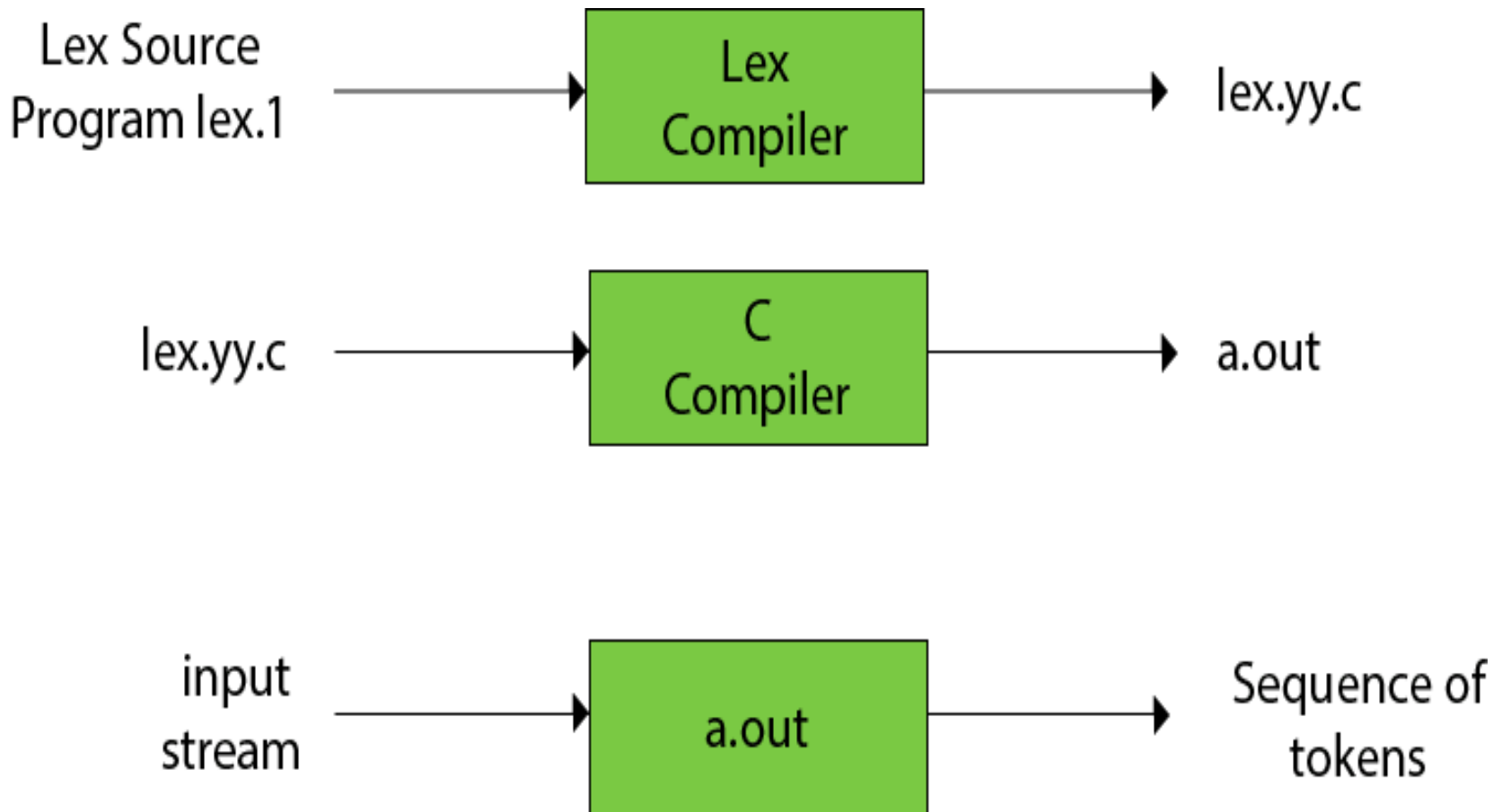
LEX

- Lex is a program that generates lexical analyzer. It is used with YACC parser generator.
- The lexical analyzer is a program that transforms an input stream into a sequence of tokens.

The function of Lex is as follows:

- Lex file format

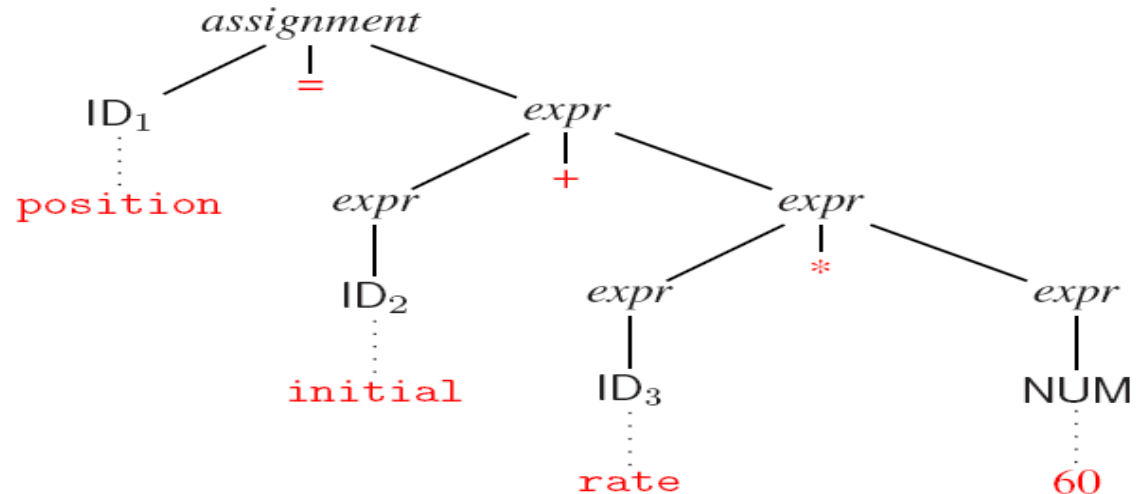
{definitions} %% {rules} %% {user subroutines}



Syntax Analysis

- The **Syntactic Analysis** is also called **Parsing**.
- Tokens are grouped into grammatical phrases represented by a **Parse Tree**
- The hierarchical structure is expressed by recursive rules, called *Productions*.

Parse Tree: An Example



Semantic Analysis

- The **Semantic Analysis** phase checks the program for semantic errors (**Type Checking**) and gathers type information for the successive phases.
- **Type Checking:** Check types of operands .No real number as index for array; etc.

Intermediate Code Generation

- ❑ An intermediate code is generated as a program for an abstract machine.
- ❑ The intermediate code should be easy to translate into the target program.
- ❑ As intermediate code we consider the *three-address code*, similar to assembly:
 - sequence of instructions with at most *three* operands such that:
 - There is at most one operator, in addition to the assignment.
 - make explicit the operators precedence.
 - Temporary names must be generated to compute intermediate operations.

Code Optimization

- This phase attempts to improve the intermediate code so that faster-running
 - machine code can be obtained.
- Different compilers adopt different optimization techniques.

Code Generation

- This phase generates the target code consisting of assembly code.
 1. Memory locations are selected for each variable;
 2. Instructions are translated into a sequence of assembly instructions;
 3. Variables and intermediate results are assigned to memory registers.

The Structure of a Compiler

SYMBOL TABLE

1	position	...
2	initial	...
3	rate	...
4		



```
position := initial + rate * 60
```

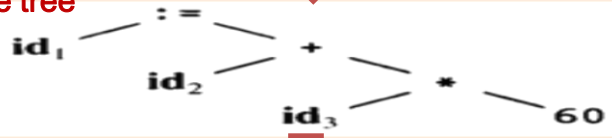
Scanner
[Lexical Analyzer]

Tokens

```
id1 := id2 + id3 * 60
```

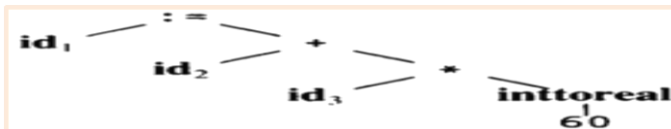
Parser
[Syntax Analyzer]

Parse tree



Semantic Process
[Semantic analyzer]

Abstract Syntax Tree w/ Attributes



Code Generator
[Intermediate Code Generator]

Non-optimized Intermediate Code

```
temp1 := inttoreal(60)
temp2 := id3 * temp1
temp3 := id2 + temp2
id1 := temp3
```

Code Optimizer

Optimized Intermediate Code

```
temp1 := id3 * 60.0
id1 := id2 + temp1
```

Code Optimizer

Target machine code

```
MOVF id3, R2
MULF #60.0, R2
MOVF id2, R1
ADDF R2, R1
MOVF R1, id1
```

Symbol Table Management

- A symbol table is a data structure containing a record for each identifier, with fields for the attributes of the identifier.
- The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly.

Error Detection and Reporting

- Each phase encounter errors
 - The lexical phase can detect errors where the characters remaining in the input do not form any token of the language.
 - Errors where the token stream violates the structure rules (syntax) of the language are determined by the syntax analysis phase.
 - During semantic analysis the compiler tries to detect constructs that have the right syntactic structure but no meaning to the operation involved
 - e.g., if we try to add two identifiers, one of which is the name of an array, and the other the name of a procedure

Compiler-Construction Tools

Other names are

- ✓ Compiler – compiler
- ✓ Compiler – generators
- ✓ Translator – writing system

For compiler construction there are special tools like :

- 1- Scanner generators Eg. Lex and Flex
- 2- Parser generators Eg. Yacc and Bison
- 3- Syntax-directed translation engines
- 4- Code-generator generators
- 5- Data-flow analysis engines